LEADERS UNIVERSITY / LICENCE BI

Aide-mémoire Git · Codex · BI

Le cycle Git, les gestes GitHub, les habitudes Codex, le vocabulaire BI et les contrôles · à garder ouvert pendant les séances

15	6	5
GESTES DU CYCLE GIT	MOTS CODEX À DISTINGUER	PREUVES DE SÉANCE

SOLEN DIGITAL · CLIENT FICTIF MAISON ZITOUNA

Branche → demande → contrôle → diff → PR → revue → fusion

06 / AIDE-MÉMOIRE

Youssef Douma · Leaders University

Édition pédagogique · 28 octobre 2026 · données fictives

06 / AIDE-MÉMOIRE · GIT

Git · le cycle, dans l'ordre

A = auteur, B = relecteur. <…> = à remplacer, chevrons compris. Terminal Ubuntu, dans `~/solen-bi`, venv actif. Les rôles alternent à chaque PR ; chaque PR ajoute au journal **état, décision, preuve, prochain objectif**.

# · qui	Commande ou geste	Pourquoi
1 · A	`cd ~/solen-bi` puis `source .venv/bin/activate`	bon dossier, bon Python
2 · A	`git switch main` puis `git pull`	partir du dernier `main`
3 · A	`git switch -c <branche>` (ex. `rejets-motifs`)	travailler à l'écart de `main`
4 · A	demande à Codex (p. 5), avec une prévision	un petit objectif à la fois
5 · A	lancer les contrôles (p. 8)	une preuve, pas une promesse
6 · A	`git status` puis `git diff`	voir chaque ligne changée
7 · A	`git add <fichier>` puis `git commit -m "<message>"`	une photo nommée
8 · A	`git push -u origin <branche>` (ensuite : `git push`)	envoyer la branche

# · qui	Commande ou geste	Pourquoi
9 · A	**Compare & pull request → Create pull request**	demander une revue
10 · B	`git fetch` puis `git switch <branche>`	récupérer la branche
11 · B	contrôles + `$revoir-bi` (lecture seule) + sa lecture	vérifier sans modifier
12 · B	**Files changed → Review changes → Approve / Request changes**	une revue écrite
13 · A	corrections, même branche : `git commit`, `git push`	la PR se met à jour
14 · B	CI verte + approbation → **Merge pull request → Confirm merge**	entrer dans `main`
15 · A+B	`git switch main && git pull` · `git branch -d <branche>`	réaligner les copies

06 / AIDE-MÉMOIRE · GIT

Git · lire avant d'agir, réparer sans casser

Commande	Ce qu'elle montre
`pwd`	le dossier courant (attendu : `/home/<vous>/solen-bi`)
`git status`	branche, fichiers modifiés ou nouveaux, ce qui est prêt à commiter
`git diff` · `git diff --staged`	lignes changées (+ ajoutée, - retirée) · ce qui partira au commit
`git log --oneline -5`	les 5 derniers commits
`git branch`	branches locales ; * = branche courante

Interdits

`git push --force`, `git reset --hard`, supprimer `.git`. Doute : `git status`, `git log --oneline -5`, on montre.

Symptôme	Réparer
`fatal: not a git repository`	`cd ~/solen-bi`
commit fait sur `main`	`git switch -c <branche>`, push, PR ; après fusion : `git switch main && git pull`
`Your local changes … would be overwritten`	commit sur la bonne branche, ou `git stash` → changer de branche → `git stash pop`
push refusé, `protected branch`	normal : branche + PR
push refusé, `fetch first`	`git pull` sur la branche, contrôles, `git push`
PR `out-of-date`	sur la branche : `git fetch`, `git merge origin/main`, contrôles, push
`Not possible to fast-forward`	ne rien forcer ; montrer `git status` et `git log --oneline -5`
check rouge sur la PR	**Details**, lire la fin, relancer chez soi, corriger le code (jamais contrôles ni référence)
`CONFLICT`	lire les deux versions, décider à deux, retirer `<<<<<<<` `=======` `>>>>>>>`, contrôles, `git add`, `git commit` ; annuler : `git merge --abort`

06 / AIDE-MÉMOIRE · GITHUB

Gestes GitHub

Geste	Où cliquer	À retenir
Créer le dépôt (A)	**+ → New repository**	public, sans README
Inviter (A)	**Settings → Collaborators → Add people**	B accepte l'invitation
Protéger `main` (A)	**Settings → Branches → Add classic branch protection rule** : `main`	PR requise, 1 approbation, **Do not allow bypassing the above settings**
Ouvrir une PR	**Compare & pull request**, **Reviewers**, **Create pull request**	titre court ; description quoi / pourquoi / preuve / à relire
Relire	**Files changed** → **+** sur une ligne → **Review changes**	citer un fichier et une ligne ; ce qui est vérifié, ce qui manque. « OK » seul n'est pas une revue.
Décider	**Approve** ou **Request changes** → **Submit review**	l'auteur ne peut pas approuver sa propre PR
Lire la CI	onglet **Checks** ou **Details** sur la croix rouge	la CI rejoue les contrôles sur une machine Linux neuve ; vert avant de fusionner
Fusionner	**Merge pull request → Confirm merge**	jamais sa propre PR sans approbation
Répondre à une revue	sous chaque commentaire	« corrigé dans `<commit>` » ou « non retenu parce que… » avec la preuve

Description de PR · modèle proposé par $preparer-pr

```
Titre : Rejeter les lignes à quantité invalide
## Ce qui change
- solen/nettoyage.py : motif quantite_invalide
## Preuve
- python -m controles.verifier --etape initial → <résultat réel>
## À relire en priorité
- <fichier:ligne> — <pourquoi>
## Non vérifié
- <ce qui n'a pas été contrôlé>
```

Secrets

Aucun mot de passe, jeton ou clé dans un fichier, un commit, une URL ou une demande à Codex. Connexion : `gh auth login` (navigateur) ; `gh auth status` pour vérifier.

06 / AIDE-MÉMOIRE · CODEX

Habitudes Codex

Une bonne demande	Exemple Solen
Un petit objectif observable	« compter les lignes du CSV de ventes du lot 1 »
Les fichiers concernés, et ceux à ne pas toucher	code dans `solen/` ; jamais `tronc-commun/`, `reference/`, `controles/`
Le contrôle attendu et **votre prévision**	« `--etape initial` doit afficher `fact_vente : 1960 lignes` »
Plan d'abord si plus d'un fichier	« Propose ton plan en 3 étapes, attends mon accord » (ou `/plan`)

Puis : **lire le diff** (onglet de revue Ctrl + Maj + G, ou `git diff`), **relancer soi-même** les contrôles, **écrire une note courte** au journal. Une commande annoncée par Codex n'est pas une preuve tant que vous ne l'avez pas lancée. Un contrôle **déterministe** donne toujours le même résultat : un compte, une somme en millimes, des rejets par motif.

Mot	Sens	Geste
Modèle	GPT-6 Luna, annoncé pour Free (application de bureau, selon le déploiement)	sélecteur de modèle, Ctrl + Maj + M
Effort	combien Codex réfléchit : Light, Medium, High, Extra High (sous Advanced)	High pour commencer ; Light pour une question courte ; Extra High si un débogage reste bloqué
Vitesse	« Standard » : vitesse de service, pas un effort	rien à régler
Token	morceau de texte ; demandes, fichiers lus et réponses se comptent en tokens	peu de fichiers à la fois
Contexte	ce que Codex garde en tête dans cette conversation	conversation trop longue : note d'état (prompt 16) + conversation neuve
Quota	allocation d'usage sur une période ; elle varie ; aucune durée promise	`/status` avant une grosse demande ; limite : commit, noter l'heure de reprise, continuer sans Codex

06 / AIDE-MÉMOIRE · CODEX

Revue, fichiers guides, travail sans Codex

Revue : lecture seule → constater → corriger → recontrôler

Étape	Ce que vous faites
1. Lire	« Utilise $revoir-bi. Branche : <branche>. Lecture seule. Au plus 3 défauts établis : fichier, ligne, exemple observable. » (prompt 11)
2. Choisir	Vous décidez quels constats traiter ; une demande de correction par constat (prompt 12).
3. Recontrôler	Relancer **tous** les contrôles, relire le diff de la correction, tester un cas limite (vente NBE du 2026-07-14 et du 2026-07-15).
4. Écrire	Votre revue GitHub dit ce que **vous** avez reproduit. « Aucun défaut établi » n'est pas « tout est juste ».

Les fichiers qui guident Codex

Fichier	Rôle
`AGENTS.md`	règles du dépôt, lues par Codex avant tout travail. Vérifier : « Quelles instructions as-tu lues ? »
`.agents/skills/<nom>/SKILL.md`	une méthode réutilisable (`name` + `description`) : `$expliquer-bi`, `$revoir-bi`, `$preparer-pr`. Non reconnu : « Lis .agents/skills/<nom>/SKILL.md et applique sa méthode. »
Sous-agents (travailleurs parallèles)	facultatifs, seulement si vous les demandez ; surtout pour de la lecture ; ils consomment plus de tokens. Sans eux : deux conversations successives.

Codex ne fait jamais à votre place

`git commit`, `git push`, ouvrir ou fusionner une PR, écrire votre revue, modifier les sources, les références ou `controles/`. Si une réponse le propose : refusez et reformulez.

Codex indisponible ou quota atteint

Continuez : Git, revue du binôme, lecture du diff, calcul à la main sur l'échantillon, interprétation, préparation de la prochaine demande. Point de reprise : `modele-binome/` du dépôt du cours + votre `notes/JOURNAL.md`. Jamais de code à la main, jamais d'achat : attendez le reset affiché.

06 / AIDE-MÉMOIRE · BI

Vocabulaire BI · Solen Commerce 360

Maison Zitouna (fictif), boutiques **TUN**, **NBE**, **SFX**. Montants en **millimes entiers** (1 TND = 1 000 millimes), affichés en TND à 3 décimales. Chiffres : `tronc-commun/reference/`.

Mot	Définition courte	Exemple Solen
ETL	extraire, transformer, charger	`ventes.csv` → nettoyage → `outputs/entrepot.sqlite`
Grain	ce que représente une ligne de fait	une ligne de ticket : `E-001-1`, 2 × P01 à 18,500 TND = 37 000 millimes
Fait	événement mesuré, avec montants	`fact_vente`, `fact_retour`
Dimension	ce qui décrit le fait	`dim_boutique` (zone), `dim_produit`, `dim_date`
Clé métier	identifiant réel, en texte	`boutique_id = "NBE"`
Clé technique	entier créé par l'entrepôt	`boutique_key` : une par version de NBE
Étoile	faits au centre, reliés aux dimensions par les `*_key`	`fact_vente` → `dim_date`, `dim_produit`, `dim_boutique`
Rejet	ligne écartée + motif, dans `rejets.csv`	`E-006-1` en double (2 puis 4 savons) → `id_conflit` ; lot 1 : 26 rejets

Mot	Définition courte	Exemple Solen (lot 1)
Ventes brutes	Σ quantité × prix du ticket	78 067,830 TND
Ventes nettes	ventes brutes − retours	78 067,830 − 3 758,330 = 74 309,500 TND
Marge sur coût d'achat	ventes nettes − coût d'achat des unités nettes (jamais « bénéfice »)	74 309,500 − 45 317,800 = 28 991,700 TND
Taux de retour	retours / ventes brutes	3 758,330 / 78 067,830 = 4,81 %
SCD2	historique par versions datées	NBE passe de Cap Bon à Nord-Est le 2026-07-15 : vente de juin → Cap Bon, d'août → Nord-Est
Incrément	charger seulement le nouveau lot	lot 2 : `fact_vente` passe de 1 960 à 2 345 lignes
Rejeu	relancer le même lot sans doublon	2e `increment` : `fact_vente` reste à 2 345

06 / AIDE-MÉMOIRE · VÉRIFIER

Contrôles et preuves de séance

Terminal · dans ~/solen-bi, venv actif

```
source .venv/bin/activate        # à chaque nouveau terminal
python -m solen initial          # puis increment, export, app
python -m controles.verifier --mission $(cat mission.txt) --etape initial
                                 # étapes : initial, increment, rejeu,
                                 #          export, app, tout
pytest controles                 # tests fournis
pytest -q                        # tous les tests, y compris les vôtres
```

Étape du vérificateur	Ce qu'elle compare	Lancer avant
`initial`	tables, lignes, clés, rejets par motif, indicateurs après le lot 1	`python -m solen initial`
`increment`	mêmes contrôles après le lot 2 + versions SCD2 aux dates frontières	`initial` puis `increment`
`rejeu`	un deuxième `increment` ne change rien	`increment` une deuxième fois
`export · app`	CSV + `manifest.json` cohérents avec l'entrepôt · `index.html` autonome	`python -m solen export · app`
`tout`	détecte le lot chargé et contrôle tout ce qui existe	l'ensemble

Sortie : `OK — …` ou `ÉCHEC — <contrôle> : attendu X, observé Y`. Corrigez le code, jamais le vérificateur ni la référence. CI : `ETAPE_CI` dans `controles.yml`.

Séance	La preuve visible en fin de séance
S1	Deux comptes collaborent : dépôt binôme créé, partenaire invité, premier cycle de PR (README + journal) expliqué ; échantillon calculé à la main.
S2	`outputs/rejets.csv` ; comptes acceptés / rejetés = référence ; PR de l'auteur A relue par B ; une correction.
S3	Entrepôt à 4 tables, faits reliés ; totaux = référence ; mission du catalogue choisie ; PR de l'auteur B.
S4	Lot 2 chargé, historique préservé, rejeu stable, CI verte, conflit résolu ; Skills locaux utilisés.
S5	Application locale + rapport Power BI actualisé ; recommandation à la direction ; répétition orale.